

5 comprehending data

Work that data!

Life could be so much easier if only she'd let me help her extract, sort, and comprehend her data...



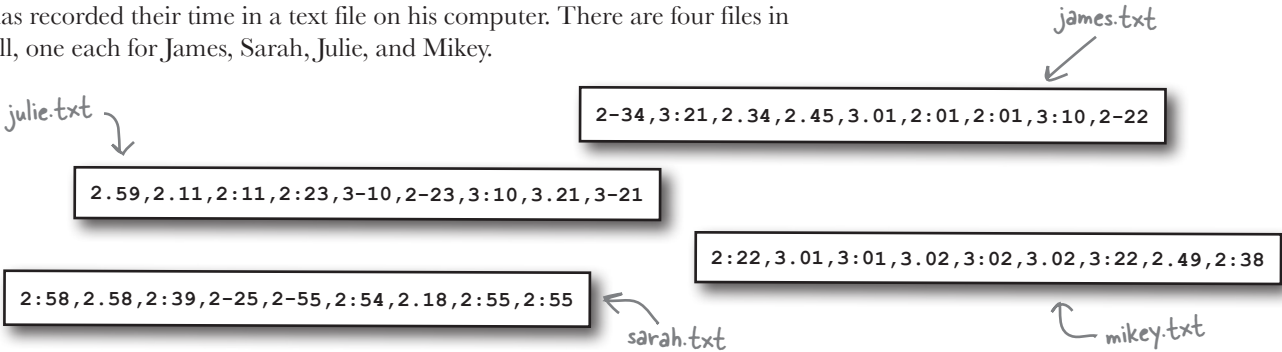
Data comes in all shapes and sizes, formats and encodings.

To work effectively with your data, you often have to manipulate and transform it into a common format to allow for efficient processing, sorting, and storage. In this chapter, you'll explore Python goodies that help you work your data up into a sweat, allowing you to achieve data-munging greatness. So, flip the page, and let's not keep the coach waiting...

Coach Kelly needs your help



The coach is an old friend, and you'd love to help. His crack squad of U10 athletes has been training hard. With each 600m run they do, Coach Kelly has recorded their time in a text file on his computer. There are four files in all, one each for James, Sarah, Julie, and Mikey.



Initially, the coach needs a quick way to know the *top three fastest times* for each athlete.

Can you help?

Do this!

Before proceeding with this chapter, take a few moments to download the four data files from the *Head First Python* support website.



Exercise

Let's begin by reading the data from each of the files into its own list. Write a short program to process each file, creating a list for each athlete's data, and display the lists on screen.

Hint: Try splitting the data on the commas, and don't forget to strip any unwanted whitespace.

Write your
code here.



Exercise Solution

Let's begin by reading the data from each of the files into its own list. You were to write a short program to process each file, creating a list for each athlete's data. You were then to display the lists on screen.

Open each of the data files in turn, read the line of data from the file, and create a list from the line of data.

Open the file.

```
with open('james.txt') as jaf:
```

Read the line of data.

```
    data = jaf.readline()
```

Convert the data to a list.

```
    james = data.strip().split(',')
with open('julie.txt') as juf:
```

```
    data = juf.readline()
```

```
    julie = data.strip().split(',')
with open('mikey.txt') as mif:
```

```
    data = mif.readline()
```

```
    mikey = data.strip().split(',')
with open('sarah.txt') as saf:
```

```
    data = saf.readline()
```

```
    sarah = data.strip().split(',')

print(james)
print(julie)
print(mikey)
print(sarah)
```

Display the four lists on screen.

there are no Dumb Questions

Q: That `data.strip().split(',')` line looks a little weird. Can you explain what's going on?

A: That's called *method chaining*. The first method, `strip()`, is applied to the line in `data`, which removes any unwanted whitespace from the string. Then, the results of the stripping are processed by the second method, `split(',')`, creating a list. The resulting list is then applied to the target identifier in the previous code. In this way, the methods are **chained together** to produce the required result. It helps if you read method chains from left to right.



Test Drive

Load your code into IDLE and run it to confirm that it's all OK for now:

Here's your
program as
displayed in
IDLE.

```
coach.py - /Users/barryp/HeadFirstPython/chapter5/coach.py
with open('james.txt') as jaf:
    data = jaf.readline()
    james = data.strip().split(',')

with open('julie.txt') as juf:
    data = juf.readline()
    julie = data.strip().split(',')

with open('mikey.txt') as mif:
    data = mif.readline()
    mikey = data.strip().split(',')

with open('sarah.txt') as saf:
    data = saf.readline()
    sarah = data.strip().split(',')

print(james)
print(julie)
print(mikey)
print(sarah)
```

And here's the output
produced by running
your code.

```
Python Shell
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
['2-34', '3:21', '2.34', '2.45', '3.01', '2:01', '2:01', '3:10', '2-22']
['2.59', '2.11', '2:11', '2:23', '3-10', '2-23', '3:10', '3.21', '3-21']
['2:22', '3.01', '3:01', '3.02', '3:02', '3.02', '3:22', '2.49', '2:38']
['2:58', '2.58', '2:39', '2-25', '2-55', '2:54', '2.18', '2:55', '2:55']
>>> |
```

Ln: 10 Col: 4

So far, so good. Coach Kelly's data is now represented by four lists in Python's memory. Other than the use of method chaining, there's nothing much new here, because you've pretty much mastered reading data from files and using it to populate lists.

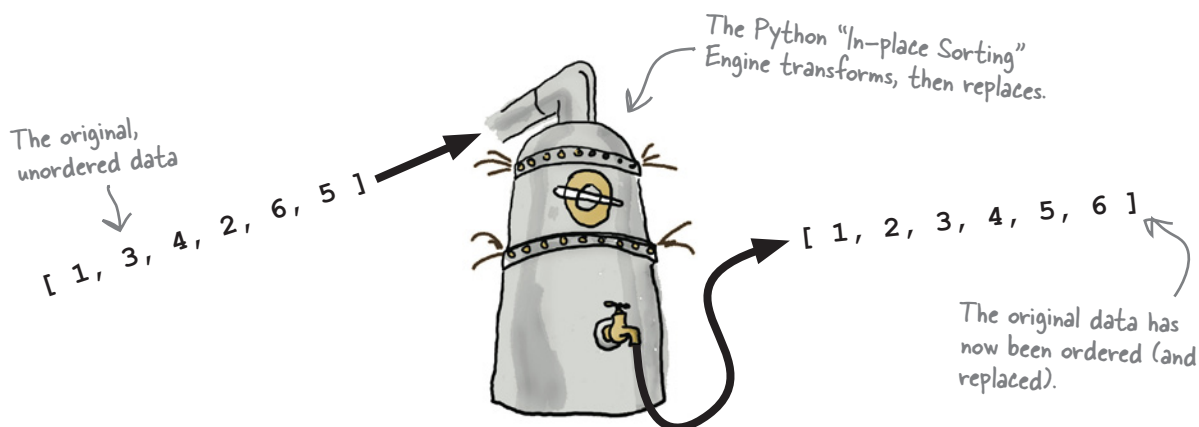
There's nothing to show the coach yet, so no point in disturbing him until his data is arranged in ascending order, which requires you to *sort* it.

Let's look at your sorting options in Python.

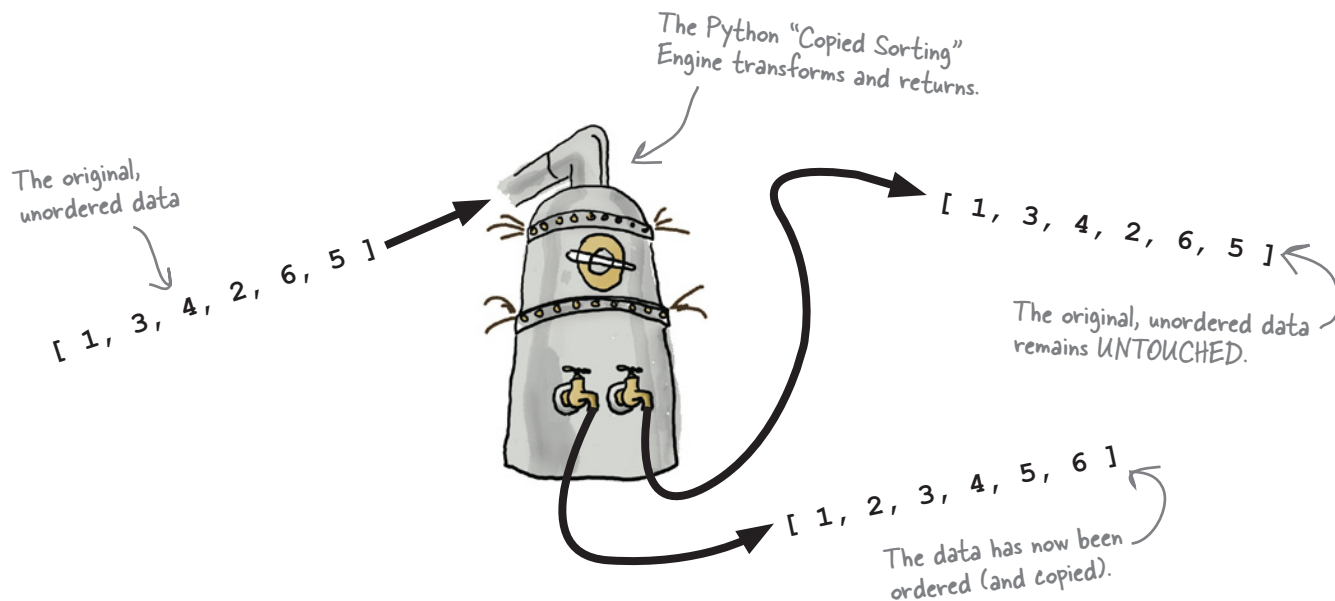
Sort in one of two ways

When it comes to sorting your data using Python, you have two options.

In-place sorting takes your data, *arranges* it in the order you specify, and then *replaces* your original data with the sorted version. The original ordering is lost. With lists, the `sort()` method provides in-place sorting:



Copied sorting takes your data, arranges it in the order you specify, and then *returns a sorted copy* of your original data. Your original data's ordering is maintained and only the copy is sorted. In Python, the `sorted()` BIF supports copied sorting.





An IDLE Session

Let's see what happens to your data when each of Python's sorting options is used. Start by creating an unordered list at the IDLE shell:

```
>>> data = [6, 3, 1, 2, 4, 5]
>>> data
[6, 3, 1, 2, 4, 5]
```

} Create a list of unordered data and assign to a variable.

Perform an in-place sort using the `sort()` method that is built in as standard to every Python list:

```
>>> data.sort()
>>> data
[1, 2, 3, 4, 5, 6]
```

← Perform *IN-PLACE* sorting on the data.

← The data's ordering has changed.

Reset data to its original unordered state, and then perform a copied sort using the `sorted()` BIF:

```
>>> data = [6, 3, 1, 2, 4, 5]
>>> data
[6, 3, 1, 2, 4, 5]
>>> data2 = sorted(data)
>>> data
[6, 3, 1, 2, 4, 5]
>>> data2
[1, 2, 3, 4, 5, 6]
```

← The data's ordering has been reset.

← Perform *COPIED* sorting on the data.

← Same as it ever was.

← The copied data is ordered from lowest to highest.



Sharpen your pencil

Either sorting option works with the coach's data, but let's use a copied sort for now to arrange to sort the data on output. In the space below, provide four amended `print()` statements to replace those at the bottom of your program.

```
.....
.....
.....
.....
.....
.....
```



Sharpen your pencil Solution

Either sorting option works with the coach's data, but let's use a copied sort for now to arrange to sort the data on output. You were to provide four amended `print()` statements to replace those at the bottom of your program.

Simply call
"sorted()" on the
data **BEFORE**
you display it on
screen.

```
print(sorted(james))
```

```
print(sorted(julie))
```

```
print(sorted(mikey))
```

```
print(sorted(sarah))
```

there are no Dumb Questions

Q: What happens to the unsorted data when I use `sort()`?

A: For all intents and purposes, it disappears. Python takes a copy, sorts it, and then replaces your original data with the sorted version.

Q: And there's no way to get the original data back?

A: No. If the ordering of the original data is important to you, use the `sorted()` BIF to transform your data into a sorted copy.



Geek Bits

You've already seen *method chaining*, and now it's time to say "hello" to *function chaining*. Function chaining allows you to apply a series of functions to your data. Each function takes your data, performs some operation on it, and then passes the transformed data on to the next function. Unlike method chains, which read from left to right, function chains read from **right to left** (just to keep things interesting).



Test Drive

Let's see if this improves your output in any way. Make the necessary amendments to your code and run it.

```

coach2.py - /Users/barryp/HeadFirstPython/chapter5/coach2.py
with open('james.txt') as jaf:
    data = jaf.readline()
    james = data.strip().split(',')

with open('julie.txt') as juf:
    data = juf.readline()
    julie = data.strip().split(',')

with open('mikey.txt') as mif:
    data = mif.readline()
    mikey = data.strip().split(',')

with open('sarah.txt') as saf:
    data = saf.readline()
    sarah = data.strip().split(',')

print(sorted(james))
print(sorted(julie))
print(sorted(mikey))
print(sorted(sarah))

```

Here's the updates to the code.

But look at THIS! The data is not at all sorted...which is, like, weird.

```

Python Shell
===== RESTART =====
>>>
>>>
[['2-22', '2-34', '2.34', '2.45', '2:01', '2:01', '3.01', '3:10', '3:21'],
 ['2-23', '2.11', '2.59', '2:11', '2:23', '3-10', '3-21', '3.21', '3:10'],
 ['2.49', '2:22', '2:38', '3.01', '3.02', '3.02', '3:01', '3:02', '3:22'],
 ['2-25', '2-55', '2.18', '2.58', '2:39', '2:54', '2:55', '2:55', '2:58']]
>>>

```

Look at this: 2-55 is coming BEFORE 2.18...now that is weird.

Hey, it looks like your data values are not uniform. Is the problem with all those periods, dashes, and colons?



Yes. The minute and seconds separators are confusing Python's sorting technology.

When recording his athletes' times in each of their files, Coach Kelly sometimes used a different character to separate minutes from seconds. It looks like you need to fix your data.

The trouble with time

Well...there's never enough of it, is there?

Let's look closely at the coach's data to see what the problem is. Here's Sarah *raw data* again:

```
2:58,2.58,2:39,2-25,2-55,2:54,2.18,2:55,2:55
```

sarah.txt

Oh, look: what a lovely bunch of strings...

Recall that data read from a file comes into your program *as text*, so Sarah's data looks like this once you turn it into a list of "times":

```
['2:58', '2.58', '2:39', '2-25', '2-55', '2:54', '2.18', '2:55', '2:55']
```

These are all strings, even though the coach thinks they're times.

And when you sort Sarah's data, it ends up in this order (which isn't quite what you were expecting):

```
['2-25', '2-55', '2.18', '2.58', '2:39', '2:54', '2:55', '2:55', '2:58']
```

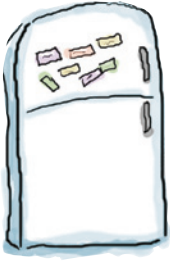
Whoops! That's not right. How can 2.18 come after 2-55?

Whoops again. 2:39 can't come between 2.58 and 2:54, can it?

I don't get what the problem is...they're all times to me.

Python sorts the strings, and when it comes to strings, a dash comes *before* a period, which itself comes *before* a colon. As all the strings start with 2, the next character in each string acts like a grouping mechanism, with the dashed times grouped and sorted, then the period times, and finally the colon times.

Nonuniformity in the coach's data is causing the sort to fail.



Code Magnets

Let's create a function called `sanitize()`, which takes as input a string from each of the athlete's lists. The function then processes the string to replace any dashes or colons found with a period and returns the sanitized string. **Note:** if the string already contains a period, there's no need to sanitize it.

Rearrange the code magnets at the bottom of the page to provide the required functionality.

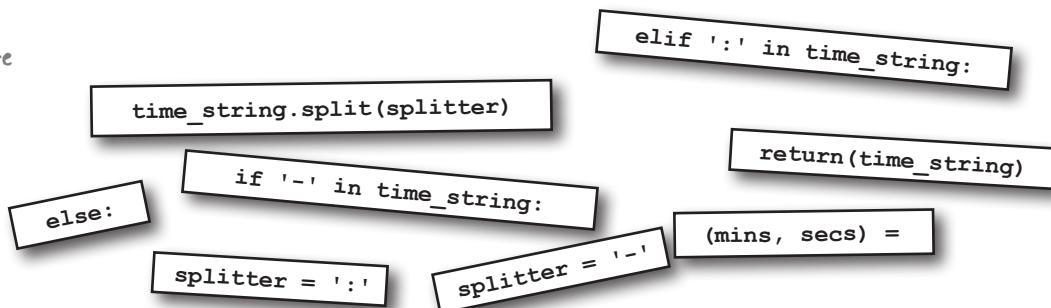
```
def sanitize(time_string):
```

```
.....
.....
.....
.....
.....
.....
.....
.....
```

```
return(mins + '.' + secs)
```

Return the sanitized time string to the caller of this function.

Your magnets are waiting.

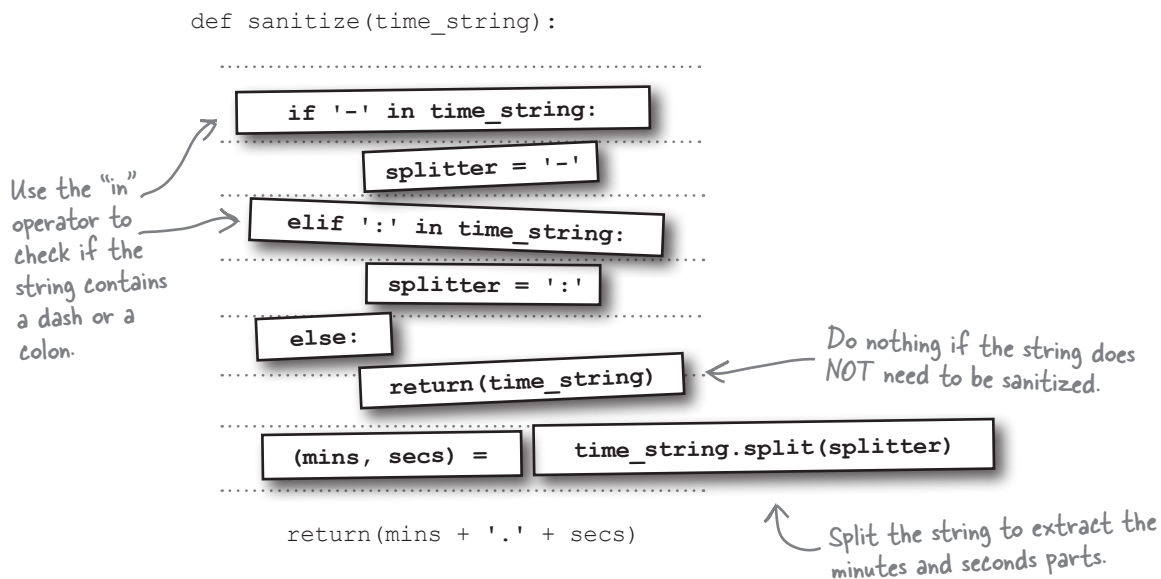




Code Magnets Solution

You were to create a function called `sanitize()`, which takes as input a string from each of the athlete's lists. The function then processes the string to replace any dashes or colons found with a period and returns the sanitized string. **Note:** if the string already contains a period, there's no need to sanitize it.

You were to rearrange the code magnets at the bottom of the previous page to provide the required functionality.



Of course, on its own, the `sanitize()` function is not enough. You need to iterate over each of your lists of data and use your new function to convert each of the athlete's times into the correct format.

Let's put your new function to work right away.



Exercise

Let's write the code to convert your existing data into a sanitized version of itself. Create four new lists to hold the sanitized data. Iterate over each athlete's list data and append each sanitized string from each list to the appropriate new list. Conclude your program by printing a sorted copy of each new list to the screen.

The code that reads the data from the data files remains unchanged (and has been compressed to fit on this page).

Add your new code here. —

```
with open('james.txt') as jaf: data = jaf.readline()
james = data.strip().split(',')
with open('julie.txt') as juf: data = juf.readline()
julie = data.strip().split(',')
with open('mikey.txt') as mif: data = mif.readline()
mikey = data.strip().split(',')
with open('sarah.txt') as saf: data = saf.readline()
sarah = data.strip().split(',')
```

This image shows a full page of white paper with horizontal dashed lines, typical of primary-ruled notebook paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

What happens to the four "print()" statements?

```
print ( ..... )
print ( ..... )
print ( ..... )
print ( ..... )
```



Exercise Solution

Let's write the code to convert your existing data into a sanitized version of itself. You were to create four new lists to hold the sanitized data. You were then to iterate over each athlete's data and append each sanitized string from each list to an appropriate new list. You were to conclude your program by printing a sorted copy of each new list to the screen.

```
with open('james.txt') as jaf: data = jaf.readline()
james = data.strip().split(',')
with open('julie.txt') as juf: data = juf.readline()
julie = data.strip().split(',')
with open('mikey.txt') as mif: data = mif.readline()
mikey = data.strip().split(',')
with open('sarah.txt') as saf: data = saf.readline()
sarah = data.strip().split(',')
```

Create four new, initially empty lists.

```
clean_james = []
clean_julie = []
clean_mikey = []
clean_sarah = []
```

```
for each_t in james:
    clean_james.append(sanitize(each_t))
for each_t in julie:
    clean_julie.append(sanitize(each_t))
for each_t in mikey:
    clean_mikey.append(sanitize(each_t))
for each_t in sarah:
    clean_sarah.append(sanitize(each_t))
```

Take each of the data items in the original lists, sanitize them, and then append the sanitized data to the appropriate new list.

The four "print()" statements now display the new lists, which are sorted, too.

```
print(sorted(clean_james))
print(sorted(clean_julie))
print(sorted(clean_mikey))
print(sorted(clean_sarah))
```



TEST DRIVE

Combine your `sanitize()` function with your amended code from the previous page, and then press F5 in IDLE to confirm the sorting is now working as expected.

The sanitized data only contains '.' as a separator.

```

Python Shell
===== RESTART =====
>>>
[ '2.01', '2.01', '2.22', '2.34', '2.34', '2.45', '3.01', '3.10', '3.21' ]
[ '2.11', '2.11', '2.23', '2.23', '2.59', '3.10', '3.10', '3.21', '3.21' ]
[ '2.22', '2.38', '2.49', '3.01', '3.01', '3.02', '3.02', '3.02', '3.22' ]
[ '2.18', '2.25', '2.39', '2.54', '2.55', '2.55', '2.55', '2.58', '2.58' ]
>>> |
  
```

The ordering works, because all the times are now comparable.

Four sorted lists

This output looks *much better*.

It's taken a bit of work, but now the data from each of the four files is both **sorted** and **uniformly formatted**. By preprocessing your data *before* you sort it, you've helped ensure Python's sorting technology performs correctly.



Geek Bits

By default, both the `sort()` method and the `sorted()` BIF order your data in **ascending** order. To order your data in **descending** order, pass the `reverse=True` argument to either `sort()` or `sorted()` and Python will take care of things for you.



Hang on a sec! Something doesn't feel quite right...look at all that duplicated code, as well as all those duplicated lists. This duplication is bad, right? Is this really the best you can do?

That's right. Duplicated code is a problem.

As things stand, your code creates four lists to hold the data as read from the data files. Then your code creates *another* four lists to hold the sanitized data. And, of course, you're iterating all over the place...

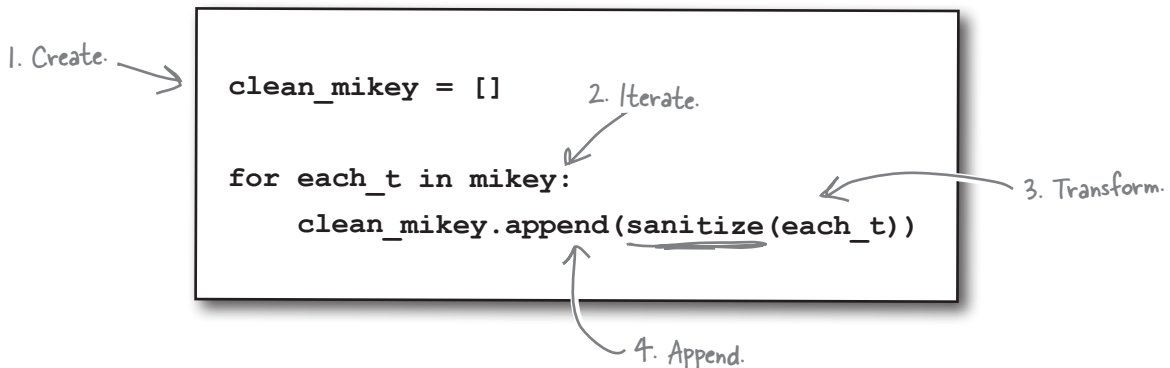
There has to be a better way to write code like this.

Transforming lists is such a common requirement that Python provides a tool to make the transformation as painless as possible. This tool goes by the rather unwieldy name of **list comprehension**. And list comprehensions are designed to reduce the amount of code you need to write when *transforming* one list into another.

Comprehending lists

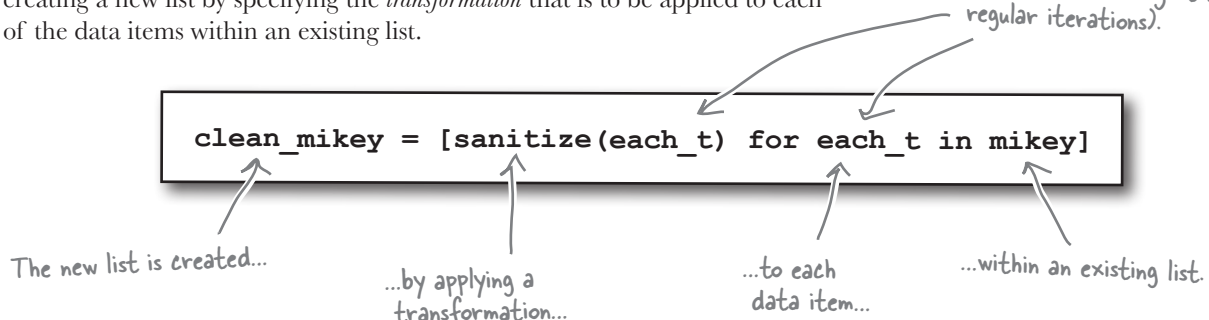
Consider what you need to do when you transform one list into another. Four things have to happen. You need to:

- 1 Create a new list to hold the transformed data.
- 2 Iterate each data item in the original list.
- 3 With each iteration, perform the transformation.
- 4 Append the transformed data to the new list.



Here's the same functionality as a **list comprehension**, which involves creating a new list by specifying the *transformation* that is to be applied to each of the data items within an existing list.

You get to pick the target identifier to use (just like with regular iterations).



What's interesting is that the transformation has been reduced to a *single line of code*. Additionally, there's no need to specify the use of the `append()` method as this action is implied within the list comprehension. Neat, eh?



An IDLE Session

Let's see some other list comprehension examples. Open up your IDLE shell and follow along with these one-liner transformations.

Start by transforming a list of minutes into a list of seconds:

```
>>> mins = [1, 2, 3]
>>> secs = [m * 60 for m in mins]
>>> secs
[60, 120, 180]
```

Simply multiply the minute values by 60.

How about meters into feet?

```
>>> meters = [1, 10, 3]
>>> feet = [m * 3.281 for m in meters]
>>> feet
[3.281, 32.81, 9.843]
```

Yes, there are 3.281 feet in a meter.

Given a list of strings in mixed and lowercase, it's a breeze to transform the strings to UPPERCASE:

```
>>> lower = ["I", "don't", "like", "spam"]
>>> upper = [s.upper() for s in lower]
>>> upper
['I', 'DON'T', 'LIKE', 'SPAM']
```

Every string comes with the "upper()" method.

Let's use your `sanitize()` function to transform some list data into correctly formatted times:

```
>>> dirty = ['2-22', '2:22', '2.22']
>>> clean = [sanitize(t) for t in dirty]
>>> clean
['2.22', '2.22', '2.22']
```

It's never been so easy to turn something dirty into something clean. 😊

It's also possible to assign the results of the list transformation back onto the original target identifier. This example transforms a list of strings into floating point numbers, and then replaces the original list data:

```
>>> clean = [float(s) for s in clean]
>>> clean
[2.22, 2.22, 2.22]
```

The "float()" B/F converts to floating point.

And, of course, the transformation can be a function chain, if that's what you need:

```
>>> clean = [float(sanitize(t)) for t in ['2-22', '3:33', '4.44']]
>>> clean
[2.22, 3.33, 4.44]
```

Combining transformations on the data items is supported, too!



Now that you know about list comprehensions, let's write four of them to process the coach's four lists of timing values. Transform each of your lists into sorted, sanitized version of themselves. Grab your pencil and in the space provided, scribble the list comprehensions you plan to use.

.....

.....

.....

.....

.....

.....

there are no Dumb Questions

Q: So...let me get this straight: list comprehensions are good and list iterations are bad, right?

A: No, that's not the best way to look at it. If you have to perform a transformation on every item in a list, using a list comprehension is the way to go, especially when the transformation is easily specified on one line (or as a function chain). List iterations can do everything that list comprehensions can, they just take more code, but iterations do provide more flexibility should you need it.



Geek Bits

Python's list comprehension is an example of the language's support for *functional programming concepts*. There's plenty of debate about the **best way** to develop program code: either procedurally, using functional programming techniques, or using object orientation. At Head First Labs, we try not to get involved in this debate, other than to rejoice in the fact that Python supports, in one way or another, all three of these programming practices.



Sharpen your pencil Solution

Now that you know about list comprehensions, you were to write four of them to process the coach's four lists of timing values. You were to transform each of your lists into sorted, sanitized version of themselves. You were to grab your pencil and in the space provided, scribble the list comprehensions you plan to use.

The list comprehension performs the transformation, and the new list is then ordered by the "sorted()" BIF.

`sorted([sanitize(t) for t in james])`

`sorted([sanitize(t) for t in julie])`

`sorted([sanitize(t) for t in mikey])`

`sorted([sanitize(t) for t in sarah])`

Rinse and repeat for the other lists.



Watch it!

Be careful about where you use the sorted() BIF when defining your list comprehensions.

You may have been tempted to use the function chain `sorted(sanitize(t))` within your list comprehension. Don't be. Recall that the transformation works on one list item at a time, **not** the entire list. In this example, the `sorted()` BIF expects to sort a list, not an individual data item.

The beauty of list comprehensions

The use of list comprehensions with the coach's athlete data has resulted in a lot less code for you to maintain. Additionally, as you get used to list comprehension syntax and usage, you'll find that their use is natural and matches the way your brain thinks about your data and the transformations that you might want to apply.

Let's confirm that your new code is working as expected.



Test Drive

Replace your list iteration code from earlier with your four new (beautiful) list comprehensions. Run your program to confirm that the results have not changed.

The screenshot shows a Python IDE with a file named `coach4.py`. The code defines a `sanitize` function and reads data from four files: `james.txt`, `julie.txt`, `mikey.txt`, and `sarah.txt`. The data is then printed using list comprehensions. A Python Shell window is open, showing the output of the program. A handwritten note points to the list comprehensions in the code, stating: "Your new list comprehensions produce EXACTLY the same output as your earlier list iterations."

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

with open('james.txt') as jaf:
    data = jaf.readline()
james = data.strip().split(',')

with open('julie.txt') as jul:
    data = jul.readline()
julie = data.strip().split(',')

with open('mikey.txt') as mik:
    data = mik.readline()
mikey = data.strip().split(',')

with open('sarah.txt') as saf:
    data = saf.readline()
sarah = data.strip().split(',')

print(sorted([sanitize(t) for t in james]))
print(sorted([sanitize(t) for t in julie]))
print(sorted([sanitize(t) for t in mikey]))
print(sorted([sanitize(t) for t in sarah]))
```

Python Shell

```
>>> ===== RESTART =====
>>>
['2.01', '2.01', '2.22', '2.34', '2.34', '2.45', '3.01', '3.10', '3.21']
['2.11', '2.11', '2.23', '2.23', '2.59', '3.10', '3.10', '3.21', '3.21']
['2.22', '2.38', '2.49', '3.01', '3.01', '3.02', '3.02', '3.02', '3.22']
['2.18', '2.25', '2.39', '2.54', '2.55', '2.55', '2.55', '2.58', '2.58']
>>>
```

Ln: 10 Col: 4

Ln: 27 Col: 0

Your new list comprehensions produce EXACTLY the same output as your earlier list iterations.

As expected, the output matches that from earlier.

You've written a program that reads Coach Kelly's data from his data files, stores his raw data in lists, sanitizes the data to a uniform format, and then sorts and displays the coach's data on screen. And all in 25 lines of code.

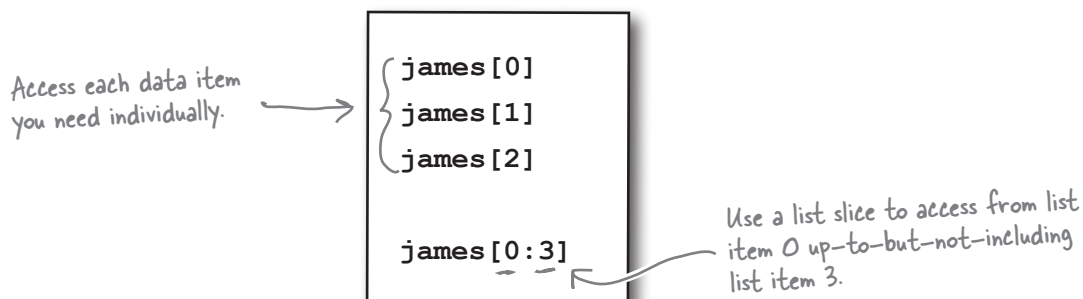
It's probably safe to let the coach take a look at your output now.

What will the coach think?



In your haste to sanitize and sort your data, you forgot to worry about what you were actually supposed to be doing: *producing the three fastest times for each athlete*. And, of course, there's no place for any duplicated times in your output.

Accessing the first three data items from any list is easy. Either specify each list item *individually* using the standard notation or use a **list slice**:



But...what about removing duplicates from your list?

Iterate to remove duplicates

Processing a list to remove duplicates is one area where a list comprehension can't help you, because duplicate removal is *not* a transformation; it's more of a *filter*. And a duplicate removal filter needs to examine the list being created *as it is being created*, which is not possible with a list comprehension.

To meet this new requirement, you'll need to revert to regular list iteration code.



Exercise

Assume that the fourth from last line of code from your current program is changed to this:

```
james = sorted([sanitize(t) for t in james])
```

That is, instead of printing the sanitized and sorted data for James to the screen, this line of code replaces James's unordered and nonuniform data with the sorted, sanitized copy.

Your next task is to write some code to remove any duplicates from the `james` list produced by the preceding line of code. Start by creating a new list called `unique_james`, and then populate it with the unique data items found in `james`. Additionally, provide code to display only the top three fastest times for James.

Hint: you might want to consider using the `not in` operator.

```
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
```



Exercise Solution

Assume that the fourth from last line of code from your current program is changed to this:

```
james = sorted([sanitize(t) for t in james])
```

That is, instead of printing the sanitized and sorted data for James to the screen, this line of code replaces James's unordered and non-uniform data with the sorted, sanitized copy.

Your next task was to write some code to remove any duplicates from the `james` list produced by the preceding line of code. You were to start by creating a new list called `unique_james` and then populate it with the unique data items found in `james`. Additionally, you were to provide code to only display the top three fastest times for James.

Create the empty list to hold the unique data items.

```
→ unique_james = []
```

```
for each_t in james:
    if each_t not in unique_james:
```

```
    unique_james.append(each_t)
```

Slice the first three data items from the list and display them on screen.

```
→ print(unique_james[0:3])
```

Do this!

Repeat the code on this page for the rest of the coach's lists: `julie`, `mikey` & `sarah`. Add all of your new code to your existing program.



Test Drive

Take all of the recent amendments and apply them to your program. Run this latest code within IDLE when you are ready.

The screenshot shows the IDLE Python editor with a file named `coach5.py`. The code defines four lists (`james`, `julie`, `mikey`, `sarah`) by sorting and sanitizing existing data. It then creates unique lists for each athlete by iterating through the sorted lists and appending only new values. Finally, it prints the first three elements of each unique list. Handwritten annotations point to the sorting/sanitizing step and the deduplication logic. Below the editor, the Python Shell shows the output of the `print` statements, displaying a 4x3 grid of time strings. A handwritten note points to the output, stating "Looking good!". The shell also shows a "RESTART" prompt and the current cursor position at line 69, column 4.

```
coach5.py - /Users/barryp/HeadFirstPython/chapter5/coach5.py

james = sorted([sanitize(t) for t in james])
julie = sorted([sanitize(t) for t in julie])
mikey = sorted([sanitize(t) for t in mikey])
sarah = sorted([sanitize(t) for t in sarah])

unique_james = []
for each_t in james:
    if each_t not in unique_james:
        unique_james.append(each_t)

print(unique_james[0:3])

unique_julie = []
for each_t in julie:
    if each_t not in unique_julie:
        unique_julie.append(each_t)

print(unique_julie[0:3])

unique_mikey = []
for each_t in mikey:
    if each_t not in unique_mikey:
        unique_mikey.append(each_t)

print(unique_mikey[0:3])

unique_sarah = []
for each_t in sarah:
```

Sort and sanitize each list.

Remove the duplicates.

```
Python Shell

>>> ===== RESTART =====
>>>
pri ['2.01', '2.22', '2.34']
    ['2.11', '2.23', '2.59']
    ['2.22', '2.38', '2.49']
    ['2.18', '2.25', '2.39']
>>> |
```

Looking good!

Ln: 69 Col: 4

It worked!

You are now displaying only the top three times for each athlete, and the duplicates have been successfully removed.

The list iteration code is what you need in this instance. There's a little bit of duplication in your code, but it's not too bad, is it?



"Not too bad" ...you're kidding, right?!?
Surely there's something that can be done
with all that duplicated duplicate code?

The irony is hard to avoid, isn't it?

The code that removes duplicates from your lists is itself duplicated.

Sometimes such a situation is unavoidable, and sometimes creating a small function to factor out the duplicated code can help. But something still doesn't feel quite right here...

Wouldn't it be dreamy if there were a way to quickly and easily remove duplicates from an existing list? But I know it's just a fantasy...



Remove duplicates with sets

In addition to lists, Python also comes with the **set** data structure, which behaves like the sets you learned all about in math class.

The overriding characteristics of sets in Python are that the data items in a set are *unordered* and *duplicates are not allowed*. If you try to add a data item to a set that already contains the data item, Python simply ignores it.

Create an empty set using the `set()` BIF, which is an example of a *factory function*:

Create a new, empty set, and assign it to a variable.

```
distances = set()
```

It is also possible to *create* and *populate* a set in one step. You can provide a list of data values between curly braces or specify an existing list as an argument to the `set()` BIF, which is the *factory function*:

```
distances = {10.6, 11, 8, 10.6, "two", 7}
```

Any duplicates in the supplied list of data values are ignored.

Any duplicates in the "james" list are ignored. Cool.

```
distances = set(james)
```

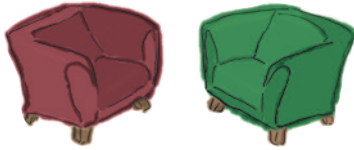
the Scholar's Corner



Factory Function: A factory function is used to make new data items of a particular type. For instance, "`set()`" is a factory function because it makes a new set. In the real world, factories make things, hence the name.



Fireside Chats



Tonight's talk: **Does list suffer from set envy?**

List:

[sings] "Anything you can do, I can do better. I can do anything better than you."

Can you spell "d-a-t-a l-o-s-s"? Getting rid of data automatically sounds kinda *dangerous* to me.

Seriously?

And that's *all* you do?

And they pay you for that?!?

Have you ever considered that I like my duplicate values. I'm very fond of them, you know.

Which isn't very often. And, anyway, I can always rely on the kindness of others to help me out with any duplicates that I don't need.

Set:

I'm resisting the urge to say, "No, you can't." Instead, let me ask you: what about handling duplicates? When I see them, I throw them away *automatically*.

But that's what I'm supposed to do. Sets aren't allowed duplicate values.

Yes. That's why I exist...to store sets of values. Which, when it's needed, is a real lifesaver.

That's all I need to do.

Very funny. You're just being smug in an effort to hide from the fact that you can't get rid of duplicates on your own.

Yeah, right. Except when you *don't* need them.

I think you meant to say, "the kindness of `set()`", didn't you?



To extract the data you need, replace all of that list iteration code in your current program with four calls to `sorted(set(...)) [0:3]`.



Head First Code Review

The Head First Code Review Team has taken your code and annotated it in the only way they know how: *they've scribbled all over it*. Some of their comments are confirmations of what you might already know. Others are suggestions that might make your code better. Like all code reviews, these comments are an attempt to improve the quality of your code.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)
```

A comment would be nice to have here.

I think we can make a few improvements here.



Meet the Head First Code Review Team.

What happens if one of these files is missing?!? Where's your exception handling code?

```
with open('james.txt') as jaf:
    data = jaf.readline()
    james = data.strip().split(',')

with open('julie.txt') as juf:
    data = juf.readline()
    julie = data.strip().split(',')

with open('mikey.txt') as mif:
    data = mif.readline()
    mikey = data.strip().split(',')

with open('sarah.txt') as saf:
    data = saf.readline()
    sarah = data.strip().split(',')
```

There's a lot going on here, but we find it's not too hard to understand if you read it from the inside out.

There's a bit of duplication here. You could factor out the code into a small function; then, all you need to do is call the function for each of your athlete data files, assigning the result to an athlete list.

```
print(sorted(set([sanitize(t) for t in james]))[0:3])
print(sorted(set([sanitize(t) for t in julie]))[0:3])
print(sorted(set([sanitize(t) for t in mikey]))[0:3])
print(sorted(set([sanitize(t) for t in sarah]))[0:3])
```

Ah, OK. We get it. The slice is applied to the list produced by "sorted()", right?



Exercise

Let's take a few moments to implement the review team's suggestion to turn those four `with` statements into a function. Here's the code again. In the space provided, create a function to abstract the required functionality, and then provide one example of how you would call your new function in your code:

```
with open('james.txt') as jaf:
    data = jaf.readline()
james = data.strip().split(',')

with open('julie.txt') as juf:
    data = juf.readline()
julie = data.strip().split(',')

with open('mikey.txt') as mif:
    data = mif.readline()
mikey = data.strip().split(',')

with open('sarah.txt') as saf:
    data = saf.readline()
sarah = data.strip().split(',')
```

Write your new
function here.



.....

.....

.....

.....

.....

.....

.....

Provide one
example call.



.....

.....



Exercise Solution

You were to take a few moments to implement the review team's suggestion to turn those four with statements into a function. In the space provided, your were to create a function to abstract the required functionality, then provide one example of how you would call your new function in your code:

```
with open('james.txt') as jaf:
    data = jaf.readline()
james = data.strip().split(',')

with open('julie.txt') as juf:
    data = juf.readline()
julie = data.strip().split(',')

with open('mikey.txt') as mif:
    data = mif.readline()
mikey = data.strip().split(',')

with open('sarah.txt') as saf:
    data = saf.readline()
sarah = data.strip().split(',')
```

Create a new function. → **def get_coach_data(filename):**

try:

with open(filename) as f: → Open the file, and read the data.

data = f.readline()

return(data.strip().split(',')) → Perform the split/strip trick on the data prior to returning it to the calling code.

except IOError as ioerr:

print('File error: ' + str(ioerr))

return(None) → Tell your user about the error (if it occurs) and return "None" to indicate failure.

Calling the function is straightforward. → **sarah = get_coach_data('sarah.txt')**

→ Provide the name of the file to process.



Test Drive

It's time for one last run of your program to confirm that your use of sets produces the same results as your list-iteration code. Take your code for a spin in IDLE and see what happens.

The screenshot shows a Python IDE with two windows. The top window, titled 'coach5.py', contains the following code:

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

with open('james.txt') as jaf:
    data = jaf.readline()
    james = data.strip().split(',')

sarah = data.strip().split(',')

print(sorted(set([sanitize(t) for t in james])[0:3]))
print(sorted(set([sanitize(t) for t in julie])[0:3]))
print(sorted(set([sanitize(t) for t in mikey])[0:3]))
print(sorted(set([sanitize(t) for t in sarah])[0:3]))
```

The bottom window, titled 'Python Shell', shows the output of the script after a 'RESTART' command:

```
>>> ===== RESTART =====
>>>
['2.01', '2.22', '2.34']
['2.11', '2.23', '2.59']
['2.22', '2.38', '2.49']
['2.18', '2.25', '2.39']
>>>
```

An arrow points from the output in the shell to a speech bubble that says: "As expected, your latest code does the business. Looking good!"

Excellent!

You've processed the coach's data perfectly, while taking advantage of the `sorted()` BIF, *sets*, and *list comprehensions*. As you can imagine, you can apply these techniques to many different situations. You're well on your way to becoming a Python *data-munging master*!



That's great work, and just what I need. Thanks! I'm looking forward to seeing you on the track soon...



Your Python Toolbox

You've got Chapter 5 under your belt and you've added some more Python techniques to your toolbox.

Python Lingo

- "In-place" sorting – transforms and then replaces.
- "Copied" sorting – transforms and then returns.
- "Method Chaining" – reading from left to right, applies a collection of methods to data.
- "Function Chaining" – reading from right to left, applies a collection of functions to data.

More Python Lingo

- "List Comprehension" – specify a transformation on one line (as opposed to using an iteration).
- A "slice" – access more than one item from a list.
- A "set" – a collection of unordered data items that contains no duplicates.



BULLET POINTS

- The `sort()` method changes the ordering of lists *in-place*.
- The `sorted()` BIF sorts most any data structure by providing *copied sorting*.
- Pass `reverse=True` to either `sort()` or `sorted()` to arrange your data in **descending** order.

- When you have code like this:

```
new_l = []
for t in old_l:
    new_l.
```

```
append(len(t))
```

rewrite it to use a **list comprehension**, like this:

```
new_l = [len(t) for t
in old_l]
```

- To access more than one data item from a list, use a slice. For example:

```
my_list[3:6]
```

 accesses the items from index location 3 up-to-but-not-including index location 6.
- Create a **set** using the `set()` factory function.

6 custom data objects

✧ ***Bundling code with data*** ✧



It's important to match your data structure choice to your data.

And that choice can make a big difference to the complexity of your code. In Python, although really useful, lists and sets aren't the only game in town. The Python **dictionary** lets you organize your data for speedy lookup by *associating your data with names*, not numbers. And when Python's built-in data structures don't quite cut it, the Python **class** statement lets you define your own. This chapter shows you how.

Coach Kelly is back (with a new file format)



I love what you've done, but I can't tell which line of data belongs to which athlete, so I've added some information to my data files to make it easy for you to figure it out. I hope this doesn't mess things up much.

The output from your last program in Chapter 5 was exactly what the coach was looking for, but for the fact that no one can tell which athlete belongs to which data. Coach Kelly thinks he has the solution: he's added identification data to each of his data files:

`Sarah Sweeney, 2002-6-17, 2:58, 2:58, 2:39, 2-25, 2-55, 2:54, 2.18, 2:55, 2:55, 2:22, 2-21, 2.22`

Sarah's full name Sarah's date of birth Sarah's timing data

This is "sarah2.txt", with extra data added.

If you use the `split()` BIF to extract Sarah's data into a list, the first data item is Sarah's name, the second is her date of birth, and the rest is Sarah's timing data.

Let's exploit this format and see how well things work.



Grab the updated files from the *Head First Python* website.



Code Magnets

Let's look at the code to implement the strategy outlined at the bottom of the previous page. For now, let's concentrate on Sarah's data. Rearrange the code magnets at the bottom of this page to implement the list processing required to extract and process Sarah's three fastest times from Coach Kelly's raw data.

Hint: the `pop()` method removes and returns a data item from the specified list location.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)
```

← The "sanitize()" function is as it was in Chapter 5.

```
def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            return(data.strip().split(','))
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)
```

← The "get_coach_data()" function is also from the last chapter.

Rearrange the magnets here.



.....

.....

.....

.....

sarah

's fastest times are: " +

(sarah_name, sarah_dob)

=

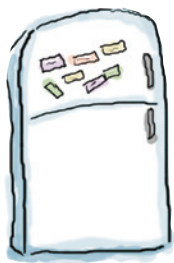
print(sarah_name +

get_coach_data('sarah2.txt')

=

sarah.pop(0), sarah.pop(0)

str(sorted(set([sanitize(t) for t in sarah]))[0:3]))



Code Magnets Solution

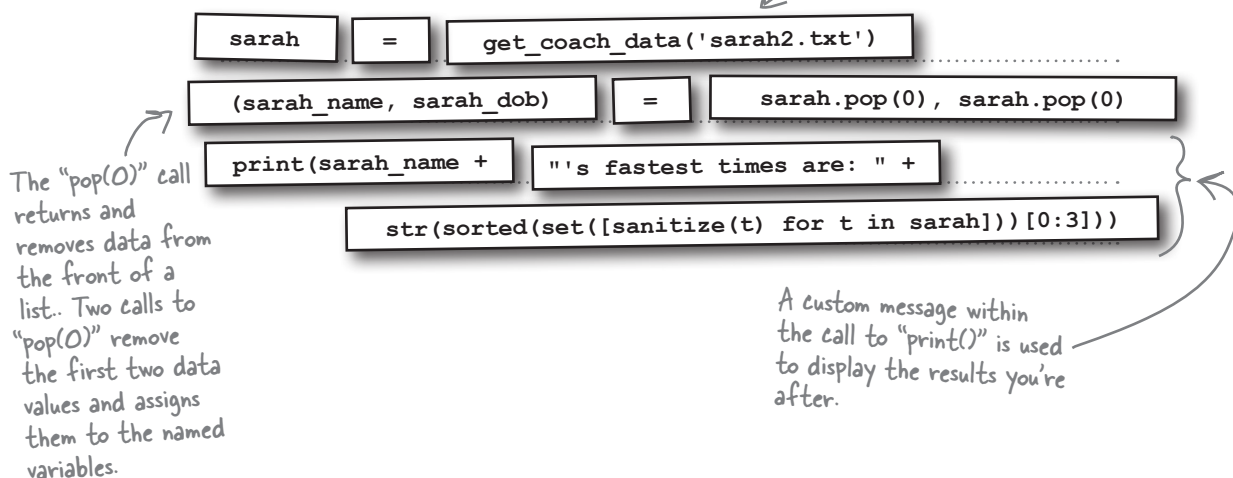
Let's look at the code to implement the strategy outlined earlier. For now, let's concentrate on Sarah's data.

You were to rearrange the code magnets at the bottom of the previous page to implement the list processing required to extract and process Sarah's three fastest times from Coach Kelly's raw data.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            return(data.strip().split(','))
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)
```

Use the function to turn Sarah's data file into a list, and then assign it to the "sarah" variable.





Test Drive

Let's run this code in IDLE and see what happens.

The screenshot shows a Python IDE window titled 'coach2.py - /Users/barryp/HeadFirstPython/chapter6/coach2.py'. The code defines two functions: `sanitize` and `get_coach_data`. `sanitize` takes a time string and returns it split by a colon or hyphen. `get_coach_data` reads a file and returns a list of time strings. The code then uses these functions to process data for Sarah Sweeney, sorting her fastest times.

Handwritten notes on the right side of the code window:

- An arrow points to the `sanitize` function with the text "Your latest code".
- An arrow points to the output line with the text "This output is much more understandable."

The Python Shell window shows the following output:

```
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']
>>> |
```

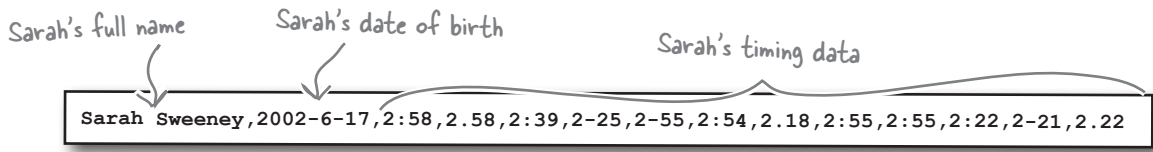
Ln: 7 Col: 4

This program works as expected, and is *fine*...except that you have to name and create Sarah's three variables in such a way that it's possible to identify *which* name, date of birth, and timing data relate to Sarah. And if you add code to process the data for James, Julie, and Mikey, you'll be up to 12 variables that need juggling. This just about works for now with four athletes. But what if there are 40, 400, or 4,000 athletes to process?

Although the data is related in "real life," within your code things are **disjointed**, because the three related pieces of data representing Sarah are stored in *three separate variables*.

Use a dictionary to associate data

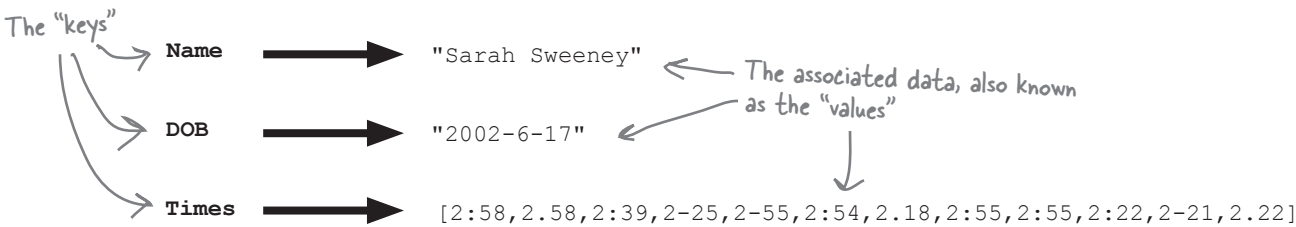
Lists are *great*, but they are not always the best data structure for every situation. Let's take another look at Sarah's data:



There's a definite **structure** here: the athlete's name, the date of birth, and then the list of times.

Let's continue to use a list for the timing data, because that still makes sense. But let's make the timing data part of *another* data structure, which associates all the data for an athlete with a single variable.

We'll use a Python **dictionary**, which *associates data values with keys*:



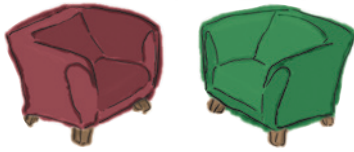
the

Scholar's Corner

Dictionary A built-in data structure (included with Python) that allows you to associate data with keys, as opposed to numbers. This lets your in-memory data closely match the structure of your actual data.



Fireside Chats



Tonight's talk: **To use a list or not to use a list?**

Dictionary:

Hi there, List. I hear you're great, but not always the best option for complex data. That's where I come in.

True. But when you do, you lose any structure associated with the data you are processing.

Isn't it always?

You *guess* so? When it comes to modeling your data in code, it's best not to guess. Be firm. Be strong. Be assertive. Use a dictionary.

[laughs] Oh, I do love your humor, List, even when you know you're on thin ice. Look, the rule is simple: *if your data has structure, use a dictionary, not a list*. How hard is that?

Which *rarely* makes sense. Knowing when to use a list and when to use a dictionary is what separates the good programmers from the great ones, right?

List:

What?!? Haven't you heard? You can put *anything* into a list, anything at all.

Well...assuming, of course, that structure is important to you.

Ummm, uh...I guess so.

That sounds like a slogan from one of those awful self-help conferences. Is that where you heard it?

Not that hard, really. Unless, of course, you are a list, and you miss being used for every piece of data in a program...

I guess so. Man, I do hate it when you're right!



Geek Bits

The Python **dictionary** is known by different names in other programming languages. If you hear other programmers talking about a "mapping," a "hash," or an "associative array," they are talking about a "dictionary."



An IDLE Session

Let's see the Python dictionary in action. Follow along with this IDLE session on your computer, ensuring that you get the same results as shown.

Start by creating two empty dictionaries, one using curly braces and the other using a factory function:

```
>>> cleese = {}
>>> palin = dict()
>>> type(cleese)
<class 'dict'>
>>> type(palin)
<class 'dict'>
```

Both techniques create an empty dictionary, as confirmed.

Add some data to both of these dictionaries by *associating values with keys*. Note the actual structure of the data is presenting itself here, as each dictionary has a Name and a list of Occupations. Note also that the palin dictionary is being created at the same time:

```
>>> cleese['Name'] = 'John Cleese'
>>> cleese['Occupations'] = ['actor', 'comedian', 'writer', 'film producer']
>>> palin = {'Name': 'Michael Palin', 'Occupations': ['comedian', 'actor', 'writer', 'tv']}
```

With your data associated with keys (which are strings, in this case), it is possible to access an individual data item using a notation similar to that used with lists:

```
>>> palin['Name']
'Michael Palin'
>>> cleese['Occupations'][-1]
'film producer'
```

Use square brackets to index into the dictionary to access data items, but instead of numbers, index with keys.

Use numbers to access a list item stored at a particular dictionary key. Think of this as "index-chaining" and read from right to left: "...the last item of the list associated with Occupations..."

As with lists, a Python dictionary can grow dynamically to store additional key/value pairings. Let's add some data about birthplace to each dictionary:

```
>>> palin['Birthplace'] = "Broomhill, Sheffield, England"
>>> cleese['Birthplace'] = "Weston-super-Mare, North Somerset, England"
```

Provide the data associated with the new key.

Unlike lists, a Python dictionary **does not maintain insertion order**, which can result in some unexpected behavior. The key point is that the dictionary *maintains the associations*, not the ordering:

```
>>> palin
{'Birthplace': 'Broomhill, Sheffield, England', 'Name': 'Michael Palin', 'Occupations': ['comedian', 'actor', 'writer', 'tv']}
>>> cleese
{'Birthplace': 'Weston-super-Mare, North Somerset, England', 'Name': 'John Cleese', 'Occupations': ['actor', 'comedian', 'writer', 'film producer']}
```

The ordering maintained by Python is different from how the data was inserted. Don't worry about it; this is OK.



Exercise

It's time to apply what you now know about Python's dictionary to your code. Let's continue to concentrate on Sarah's data for now. Strike out the code that you no longer need and replace it with new code that uses a dictionary to hold and process Sarah's data.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            return(data.strip().split(',') )
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)

sarah = get_coach_data('sarah2.txt')
(sarah_name, sarah_dob) = sarah.pop(0), sarah.pop(0)
print(sarah_name + "'s fastest times are: " +
      str(sorted(set([sanitize(t) for t in sarah]))[0:3]))
.....
.....
.....
.....
.....
.....
.....
.....
```

Strike out the code
you no longer need.



Add your dictionary
using and processing
code here.





Exercise Solution

It's time to apply what you now know about Python's dictionary to your code. Let's continue to concentrate on Sarah's data for now. You were to strike out the code that you no longer needed and replace it with new code that uses a dictionary to hold and process Sarah's data.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)
```

```
def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            return(data.strip().split(',') )
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)
```

You don't need this code anymore.

```
sarah = get_coach_data('sarah2.txt')
(sarah_name, sarah_dob) = sarah.pop(0), sarah.pop(0)
print(sarah_name + "'s fastest times are: " +
      str(sorted(set([sanitize(t) for t in sarah]))[0:3]))
```

Create an empty dictionary.

```
sarah_data = {}
```

```
sarah_data['Name'] = sarah.pop(0)
```

```
sarah_data['DOB'] = sarah.pop(0)
```

```
sarah_data['Times'] = sarah
```

Populate the dictionary with the data by associating the data from the file with the dictionary keys..

```
print(sarah_data['Name'] + "'s fastest times are: " +
```

```
      str(sorted(set([sanitize(t) for t in sarah_data['Times']]))[0:3]))
```

Refer to the dictionary when processing the data.



Test Drive

Let's confirm that this new version of your code works exactly as before by testing your code within the IDLE environment.

The screenshot shows a Python IDE window titled 'coach3.py - /Users/barryp/HeadFirstPython/chapter6/coach3.py'. The code defines two functions: `sanitize` and `get_coach_data`. `sanitize` takes a time string and returns a tuple of (mins, secs) by splitting on either a hyphen or a colon. `get_coach_data` reads a line from a file and splits it into a dictionary with 'Name', 'DOB', and 'Times' keys. Below the functions, the code calls `get_coach_data('sarah2.txt')` to get a dictionary for Sarah Sweeney, creates a new dictionary `sarah_data` with the same structure, and prints the fastest times sorted from the 'Times' list. A handwritten note with arrows points to the dictionary creation and the print statement, stating: "Your dictionary code produces the same results as earlier." Below the code is a 'Python Shell' window showing the execution output: `>>> ===== RESTART =====`, `>>>`, `Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']`, and `>>> |`. The status bar at the bottom right indicates 'Ln: 43 Col: 4'.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            return(data.strip().split(',') )
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)

sarah = get_coach_data('sarah2.txt')
sarah_data = {}
sarah_data['Name'] = sarah.pop(0)
sarah_data['DOB'] = sarah.pop(0)
sarah_data['Times'] = sarah

print(sarah_data['Name'] + "s fastest times are: " +
      str(sorted(set([sanitize(t) for t in sarah_data['Times']]))) [0:3]))
```

Python Shell

```
>>> ===== RESTART =====
>>> Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']
>>> |
```

Ln: 43 Col: 4

Which, again, works as expected...the difference being that you can now more easily determine and control which identification data **associates** with which timing data, because they are *stored in a single dictionary*.

Although, to be honest, it does take *more* code, which is a bit of a bummer. Sometimes the extra code is worth it, and sometimes it isn't. In this case, it most likely is.

Let's review your code to see if we can improve anything.



Head First Code Review

The Head First Code Review Team has been at it again: *they've scribbled all over your code*. Some of their comments are confirmations; others are suggestions. Like all code reviews, these comments are an attempt to improve the quality of your code.

```
def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)
```

```
def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            return(data.strip().split(',') )
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)
```

```
sarah = get_coach_data('sarah2.txt')
```

```
sarah_data = {}
sarah_data['Name'] = sarah.pop(0)
sarah_data['DOB'] = sarah.pop(0)
sarah_data['Times'] = sarah
```

```
print(sarah_data['Name'] + "'s fastest times are: " +
      str(sorted(set([sanitize(t) for t in sarah_data['Times']]))) [0:3]))
```

You might want to consider moving this code into the `get_coach_data()` function, too, because doing so would rather nicely abstract away these processing details. But whether you do or not is up to you. It's your code, after all!

It's great to see you taking some of our suggestions on board. Here are a few more...



Rather than building the dictionary as you go along, why not do it all in one go? In fact, in this situation, it might even make sense to do this processing within the `get_coach_data()` function and have the function return a populated dictionary as opposed to a list.

Then, all you need to do is create the dictionary from the data file using an appropriate function call, right?



Actually, those review comments are really useful. Let's take the time to apply them to your code. There are four suggestions that you need to adjust your code to support:

1. Create the dictionary all in one go.
2. Move the dictionary creation code into the `get_coach_data()` function, returning a dictionary as opposed to a list.
3. Move the code that determines the top three times for each athlete into the `get_coach_data()` function.
4. Adjust the invocations within the main code to the new version of the `get_coach_data()` function to support it's new mode of operation.

Grab your pencil and write your new `get_coach_data()` function in the space provided below. Provide the four calls that you'd make to process the data for each of the athletes and provide four amended `print()` statements:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Sharpen your pencil Solution

You were to take the time to apply the code review comments to your code. There were four suggestions that you needed to adjust your code to support:

1. Create the dictionary all in one go.
2. Move the dictionary creation code into the `get_coach_data()` function, returning a dictionary as opposed to a list.
3. Move the code that determines the top three times for each athlete into the `get_coach_data()` function.
4. Adjust the invocations within the main code to the new version of the `get_coach_data()` function to support its new mode of operation.

You were to grab your pencil and write your new `get_coach_data()` function in the space provided below, as well as provide the four calls that you'd make to process the data for each of the athletes and provide four amended `print()` statements:

```
def get_coach_data(filename):
```

```
    try:
```

```
        with open(filename) as f:
```

```
            data = f.readline()
```

1. Create a temporary list to hold the data BEFORE creating the dictionary all in one go.

```
        → templ = data.strip().split(',')
        return({'Name' : templ.pop(0),
```

```
            'DOB' : templ.pop(0),
```

```
            'Times': str(sorted(set([sanitize(t) for t in templ]))[0:3]))
```

2. The dictionary creation code is now part of the function.

```
        except IOError as ierror:
```

```
            print('File error: ' + str(ierror))
```

```
        return(None)
```

3. The code that determines the top three scores is part of the function, too.

4. Call the function for an athlete and adjust the "print()" statement as needed.

```
        james = get_coach_data('james2.txt')
```

```
        print(james['Name'] + "'s fastest times are: " + james['Times'])
```

We are showing only these two lines of code for one athlete (because repeating it for the other three is a trivial exercise).



Test Drive

Let's confirm that *all* of the re-factoring suggestions from the Head First Code Review Team are working as expected. Load your code into IDLE and take it for a spin.

```
coach3c.py - /Users/barryp/HeadFirstPython/chapter6/coach3c.py

def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            templ = data.strip().split(',')
            return({'Name': templ.pop(0),
                    'DOB': templ.pop(0),
                    'Times': str(sorted(set([sanitize(t) for t in templ]))[0:3]))}
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)

james = get_coach_data('james2.txt')
julie = get_coach_data('julie2.txt')
mikey = get_coach_data('mikey2.txt')
sarah = get_coach_data('sarah2.txt')

print(james['Name'] + "'s fastest times are: " + james['Times'])
print(julie['Name'] + "'s fastest times are: " + julie['Times'])
print(mikey['Name'] + "'s fastest times are: " + mikey['Times'])
print(sarah['Name'] + "'s fastest times are: " + sarah['Times'])
```

All of the data processing is moved into the function.

This code has been considerably tidied up and now displays the name of the athlete associated with their times.

Looking good!

```
Python Shell

>>> ===== RESTART =====
>>>
James Lee's fastest times are: ['2.01', '2.16', '2.22']
Julie Jones's fastest times are: ['2.11', '2.23', '2.59']
Mikey McManus's fastest times are: ['2.22', '2.31', '2.38']
Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']
>>> |
```

Ln: 19 Col: 4

To process additional athletes, all you need is two lines of code: the first invokes the `get_coach_data()` function and the second invokes `print()`.

And if you require additional functionality, it's no big deal to write more functions to provide the required functionality, is it?



Wait a minute...you're using a dictionary to keep your data all in one place, but now you're proposing to write a bunch of custom functions that work on your data but **aren't** associated with it. Does that really make sense?

Keeping your code and its data together is good.

It does indeed make sense to try and associate the functions with the data they are meant to work on, doesn't it? After all, the functions are only going to make sense when *related* to the data—that is, the functions will be *specific* to the data, not general purpose. Because this is the case, it's a great idea to try and bundle the code with its data.

But how? Is there an easy way to associate custom code, in the form of functions, with your custom data?

Bundle your code and its data in a class

Like the majority of other modern programming languages, Python lets you create and define an object-oriented **class** that can be used to *associate code with the data that it operates on*.



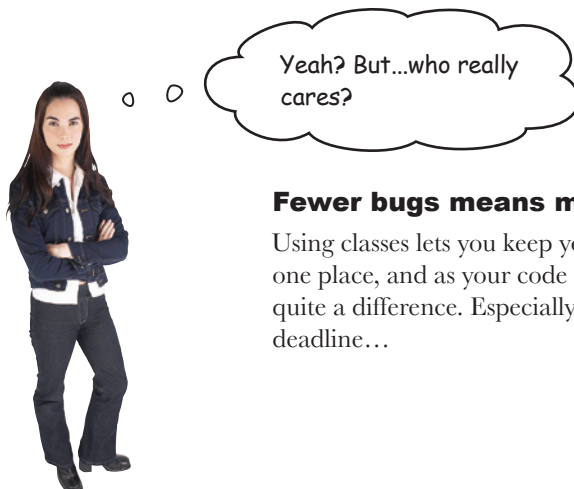
Using a class helps reduce complexity.

By associating your code with the data it works on, you reduce complexity as your code base grows.



Reduced complexity means fewer bugs.

Reducing complexity results in fewer bugs in your code. However, it's a fact of life that your programs will have functionality added over time, which will result in additional complexity. Using classes to **manage** this complexity is a *very good thing*.



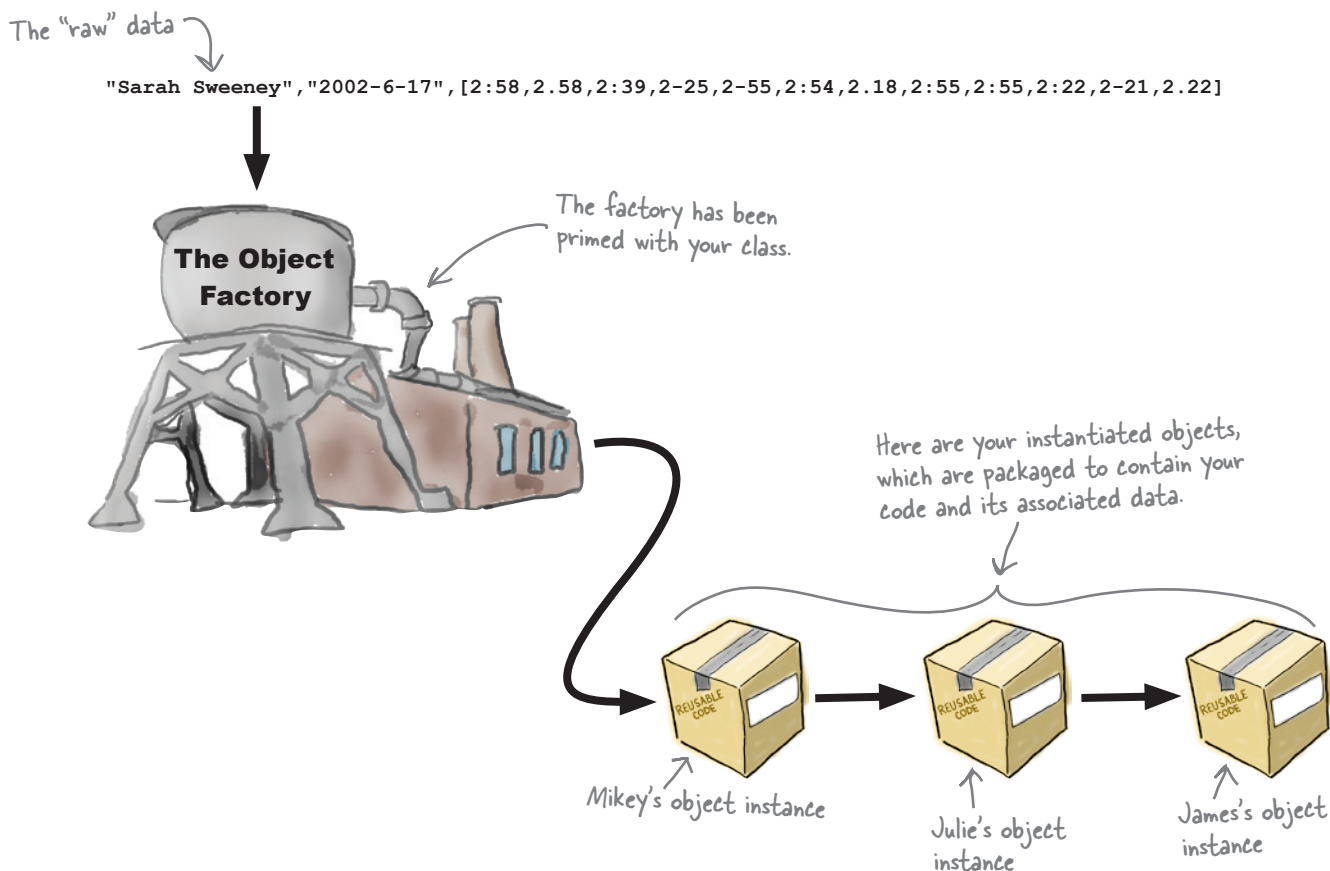
Fewer bugs means more maintainable code.

Using classes lets you keep your code and your data together in one place, and as your code base grows, this really can make quite a difference. Especially when it's 4 AM and you're under a deadline...

Define a class

Python follows the standard object-oriented programming model of providing a means for you to define the code and the data it works on as a *class*. Once this definition is in place, you can use it to create (or *instantiate*) **data objects**, which inherit their characteristics from your class.

Within the object-oriented world, your code is often referred to as the class's **methods**, and your data is often referred to as its **attributes**. Instantiated data objects are often referred to as **instances**.



Each object is **created from** the class and **shares** a similar set of characteristics. The methods (your code) are the *same* in each instance, but each object's attributes (your data) *differ* because they were created from your raw data.

Let's look at how classes are defined in Python.

Use class to define classes

Python uses **class** to create objects. Every defined class has a *special method* called `__init__()`, which allows you to control how objects are initialized.

Methods within your class are defined in much the same way as functions, that is, using `def`. Here's the basic form:

```

class Athlete:
    def __init__(self):
        # The code to initialize a "Athlete" object.
        ...
  
```

The keyword starts the definition.

Give your class a nice, descriptive name.

Don't forget the colon!

That's a double underscore before and after the word "init".

The code that initializes each object goes in here.

Creating object instances

With the class in place, it's easy to create object instances. Simply assign a *call* to the class name to each of your variables. In this way, the class (together with the `__init__()` method) provides a mechanism that lets you create a **custom factory function** that you can use to create as many object instances as you require:

```

a = Athlete()
b = Athlete()
c = Athlete()
d = Athlete()
  
```

All of these variables are unique and are "of type" Athlete.

The brackets tell Python to create a new "Athlete" object, which is then assigned to a variable.

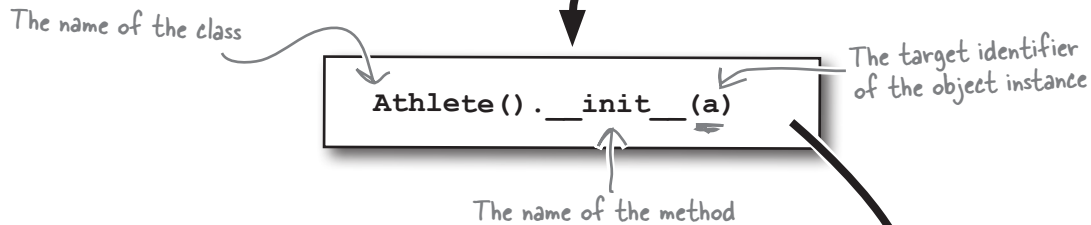
Unlike in C++-inspired languages, Python has no notion of defining a constructor called "new." Python does object construction for you, and then lets you customize your object's initial state using the `__init__()` method.

The importance of self

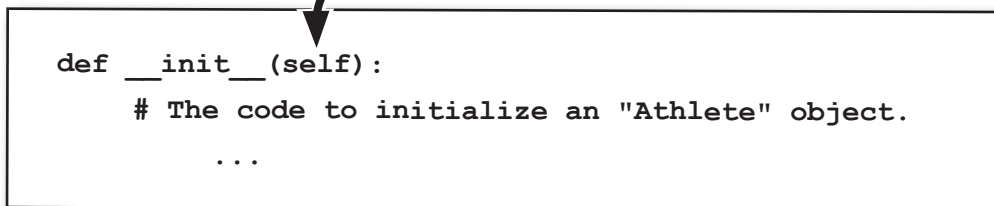
To confirm: when you define a class you are, in effect, defining a *custom factory function* that you can then use in your code to create instances:



When Python processes this line of code, it turns the factory function call into the following call, which identifies the *class*, the *method* (which is automatically set to `__init__()`), and the *object instance* being operated on:



Now take another look at how the `__init__()` method was defined in the class:



Check out what Python turns your object creation invocation into. Notice anything?

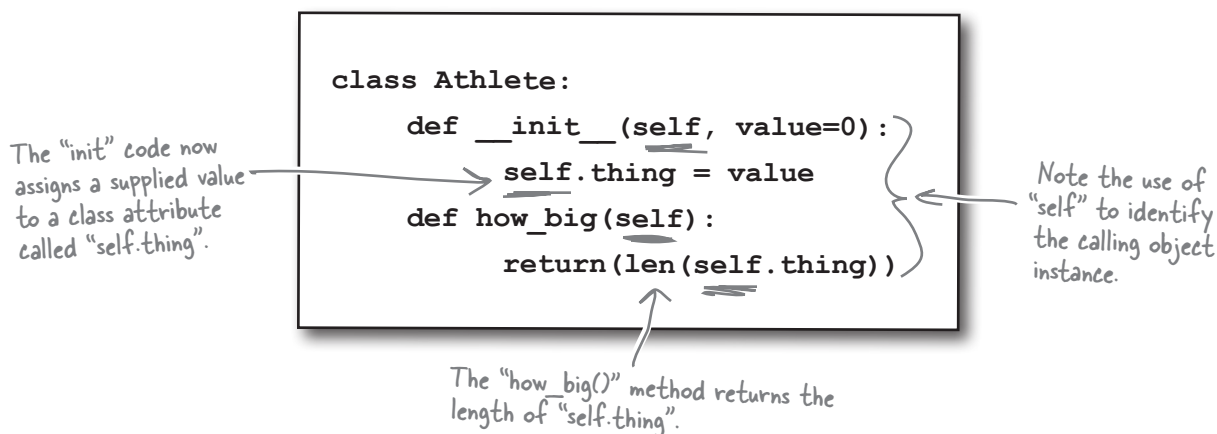
The target identifier is assigned to the **self** argument.

This is a very important argument assignment. Without it, the Python interpreter can't work out which object instance to apply the method invocation to. Note that the class code is designed to be *shared* among all of the object instances: the methods are shared, the attributes are **not**. The `self` argument helps identify which object instance's data to work on.

Every method's first argument is self

In fact, not only does the `__init__()` method require `self` as its first argument, but *so does every other method defined within your class*.

Python arranges for the first argument of every method to be the invoking (or *calling*) object instance. Let's extend the sample class to store a value in a object attribute called `thing` with the value set during initialization. Another method, called `how_big()`, returns the length of `thing` due to the use of the `len()` BIF:



When you invoke a class method on an object instance, Python arranges for the first argument to be the invoking object instance, which is *always* assigned to each method's `self` argument. This fact alone explains why `self` is so important and also why `self` needs to be the *first argument to every object method you write*:

What you write:

```
d = Athlete("Holy Grail")
```

```
d.how_big()
```

What Python executes:

```
Athlete.__init__(d, "Holy Grail")
```

```
Athlete.how_big(d)
```

The class

The method

The target identifier (or instance)



An IDLE Session

Let's use IDLE to create some object instances from a new class that you'll define. Start by creating a small class called `Athlete`:

```
>>> class Athlete:
    def __init__(self, a_name, a_dob=None, a_times=[]):
        self.name = a_name
        self.dob = a_dob
        self.times = a_times
```

Note the default values for two of the arguments.

Three attributes are initialized and assigned to three class attributes using the supplied argument data.

With the class defined, create two unique object instances which derive their characteristics from the `Athlete` class:

```
>>> sarah = Athlete('Sarah Sweeney', '2002-6-17', ['2:58', '2:58', '1:56'])
>>> james = Athlete('James Jones')
>>> type(sarah)
<class '__main__.Athlete'>
>>> type(james)
<class '__main__.Athlete'>
```

Create two unique athletes (with "james" using the default argument values).

Confirm that both "sarah" and "james" are athletes.

Even though `sarah` and `james` are both athletes and were created by the `Athlete` class's factory function, they are stored at different memory addresses:

```
>>> sarah
<__main__.Athlete object at 0x14d23f0>
>>> james
<__main__.Athlete object at 0x14cb7d0>
```

These are the memory addresses on our computer, which will differ from the values reported on yours. The key point is the memory address for "sarah" and "james" differ.

Now that `sarah` and `james` exist as object instances, you can use the familiar dot notation to access the attributes associated with each:

```
>>> sarah.name
'Sarah Sweeney'
>>> james.name
'James Jones'
>>> sarah.dob
'2002-6-17'
>>> james.dob
>>> sarah.times
['2:58', '2:58', '1:56']
>>> james.times
[]
```

The "james" object instance has no value for "dob", so nothing appears on screen.



Here's your code (except for the `sanitize()` function, which doesn't need to change). With your pencil, write code to define the `Athlete` class. In addition to the `__init__()` method, define a new method called `top3()` that, when invoked, returns the top three times.

Be sure to adjust the `get_coach_data()` function to return an `Athlete` object as opposed to a dictionary, and don't forget to amend your `print()` statements, too.

Write your `Athlete` class code here.

```
def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            templ = data.strip().split(',')
            return({'Name' : templ.pop(0),
                    'DOB'   : templ.pop(0),
                    'Times': str(sorted(set([sanitize(t) for t in templ]))[0:3]))}
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)

james = get_coach_data('james2.txt')

print(james['Name'] + "'s fastest times are: " + james['Times'])
```

What needs to change here to ensure this function returns an `Athlete` object as opposed to a dictionary?

This line of code needs to change, too.



Sharpen your pencil Solution

Here's your code (except for the `sanitize()` function, which doesn't need to change). With your pencil, you were to write code to define the `Athlete` class. In addition to the `__init__()` method, you were to define a new method called `top3()` that, when invoked, returns the top three times. You were to be sure to adjust the `get_coach_data()` function to return an `Athlete` object as opposed to a dictionary, and you weren't to forget to amend `print()`, too.

class `Athlete`:

```
def __init__(self, a_name, a_dob=None, a_times=[]):
```

```
    self.name = a_name
```

```
    self.dob = a_dob
```

```
    self.times = a_times
```

There's nothing new here as this code is taken straight from the most recent IDLE session.

```
def top3(self):
```

```
    return(sorted(set([sanitize(t) for t in self.times]))[0:3])
```

Did you remember to use "self"?

```
def get_coach_data(filename):
```

```
    try:
```

```
        with open(filename) as f:
```

```
            data = f.readline()
```

```
        templ = data.strip().split(',')

```

```
        return({'Name': templ.pop(0),
```

```
                'DOB': templ.pop(0), Athlete(templ.pop(0), templ.pop(0), templ)
```

```
                'Times': str(sorted(set([sanitize(t) for t in templ]))[0:3]))
```

```
    except IOError as ioerr:
```

```
        print('File error: ' + str(ioerr))
```

```
        return (None)
```

```
james = get_coach_data('james2.txt')
```

```
    james.name
```

```
print(james['Name'] + "'s fastest times are: " + james['Times'])
```

Remove the dictionary creation code and replace it with `Athlete` object creation code instead.

Use the dot notation to get at your data.

Invoke the "top3()" method and convert its results to a string prior to its display on screen.

```
str(james.top3())
```



Test Drive

With these changes applied to your program, let's ensure you continue to get the same results as earlier. Load your code into IDLE and run it.

```
coach4.py - /Users/barryp/HeadFirstPython/chapter6/coach4.py

class Athlete:
    def __init__(self, a_name, a_dob=None, a_times=[]):
        self.name = a_name
        self.dob = a_dob
        self.times = a_times

    def top3(self):
        return(sorted(set([sanitize(t) for t in self.times]))[0:3])

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            templ = data.strip().split(',')
            return(Athlete(templ.pop(0), templ.pop(0), templ))
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)

james = get_coach_data('james2.txt')
julie = get_coach_data('julie2.txt')
mikey = get_coach_data('mikey2.txt')
sarah = get_coach_data('sarah2.txt')

print(james.name + "s fastest times are: " + str(james.top3()))
print(julie.name + "s fastest times are: " + str(julie.top3()))
print(mikey.name + "s fastest times are: " + str(mikey.top3()))
print(sarah.name + "s fastest times are: " + str(sarah.top3()))
```

The code to the "sanitize()" function is not shown here, but it is still part of this program.

```
Python Shell

>>> ===== RESTART =====
>>>
James Lee's fastest times are: ['2.01', '2.16', '2.22']
Julie Jones's fastest times are: ['2.11', '2.23', '2.59']
Mikey McManus's fastest times are: ['2.22', '2.31', '2.38']
Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']
>>>
```

Cool! There's no change here.

And to make objects do more, I just add more methods, right?

Yes, that's correct: more functionality = more methods.

Simply add methods to **encapsulate** the new functionality you need within your class. There's no limit to how many methods a class can have, so feel free to knock yourself out!

there are no
Dumb Questions

Q: I'm not sure I see why the `top3()` method is coded to return a three-item list, as opposed to a string? Surely a string would make the `print()` statement in the main program easier to write?

A: It would, but it wouldn't be as flexible. By returning a list (albeit a small one), the `top3()` method lets the calling code decide what happens *next*, as opposed to forcing the caller to work with a string. Granted, the current program needs to treat the list like a string, but not all programs will want or need to.

Q: Why does the class even need the `top3()` method? Why not store the top three times as an attribute within the class and create it as part of the object's creation?

A: Again, better not to, because doing so is *less flexible*. If you compute and store the top three times at object creation, you make it harder to extend the list of timing data associated with the object.

For instance, if you add more timing data **after** the object is created, you'll need to arrange to recompute the top three (because the new times might be fast) and update the attribute. However, when you compute the top three times "on the fly" using a call to the `top3()` method, you always ensure you're using the most up-to-date data.

Q: OK, I get that. But, with a little extra work, I could do it during object creation, right?

A: Well, yes...but we really don't advise that. By preserving the original data in each object's attributes, you are supporting the extension of the class to support additional requirements in the future (whatever they might be). If you process the data as part of the object initialization code, the assumptions you make about how programmers will use your class might just come back to bite you.

Q: But what if I'm the only programmer that'll ever use a custom class that I write?

A: Trust us: you'll thank yourself for coding your class to be as flexible as possible when you come to use it for some other purpose in a future project. When you are creating a class, you have no idea how it will be used by other programmers in their projects. And, if you think about, you have no idea how you might use it in the future, too.

Q: OK, I think I'm convinced. But tell me: how do I go about adding more times to my existing `Athlete` objects?

A: To do more, add more methods. With your `Athlete` class created, it's a breeze to extend it to do more work for you: simply add more methods.

So, if you want to add a single new timing value to your `times` attribute, define a method called `add_time()` to do it for you. Additionally, you can add a list of times by defining a method called `add_times()`. Then all you need to do in your code is say something like this:

```
sarah.add_time('1.31')
```

to add a single time to Sarah's timing data, or say this:

```
james.add_times(['1.21', '2.22'])
```

to add a bunch of times to James's data.

Q: But surely, knowing that `times` is a list, I could write code like this to do the same thing?

```
sarah.times.append('1.31')
```

```
james.times.append(['1.21', '2.22'])
```

A: You could, but that would be a *disaster*.

Q: What?!? Why do you say that? There's nothing wrong with my suggestion, is there?

A: Well...it does indeed work. However, the **problem** with writing code like that is that it exposes (and exploits) that fact that the timing data is stored in a list within the `Athlete` class. If you later change your class implementation to use (for instance) a string *instead* of a list, you may well break all of the existing code that uses your class and that exploits the fact that the timing data is a list.

By defining your own API with `add_time()` and `add_times()`, you leave open the possibility that the way the data is stored within your class can change in the future (obviously, only if such a change makes sense). It is worth noting that one of the reasons for using object orientation is to hide away the details of a class's implementation from the users of that class. Defining your own API *directly* supports this design ideal. Exposing the internals of your class's implementation and expecting programmers to exploit it **breaks** this fundamental ideal in a very big way.



Exercise

Let's add two methods to your class. The first, called `add_time()`, appends a single additional timing value to an athlete's timing data. The second, `add_times()`, extends an athlete's timing data with one or more timing values supplied as a list.

Here's your current class: add the code to implement these two new methods.

```
class Athlete:
    def __init__(self, a_name, a_dob=None, a_times=[]):
        self.name = a_name
        self.dob = a_dob
        self.times = a_times

    def top3(self):
        return(sorted(set([sanitize(t) for t in self.times]))[0:3])
```

Add your new methods here.



.....

.....

.....

.....

.....

.....

.....



Sharpen your pencil

Don't put down the pencil just yet! Provide a few lines of code to test your new functionality:

.....

.....

.....

.....



Exercise Solution

Let's add two methods to your class. The first, called `add_time()`, appends a single additional timing value to an athlete's timing data. The second, `add_times()`, extends an athlete's timing data with one or more timing values supplied as a list.

Here's your current class: you were to add the code to implement these two new methods.

```
class Athlete:
    def __init__(self, a_name, a_dob=None, a_times=[]):
        self.name = a_name
        self.dob = a_dob
        self.times = a_times

    def top3(self):
        return(sorted(set([sanitize(t) for t in self.times]))[0:3])
```

```
def add_time(self, time_value):
    self.times.append(time_value)

def add_times(self, list_of_times):
    self.times.extend(list_of_times)
```

Don't forget to use "self"!!!

Take the supplied argument and append it to the existing list of timing values.

Take the list of supplied arguments and extend the existing list of timing values with them.



Sharpen your pencil Solution

While still holding on firmly to your pencil, you were to provide a few lines of code to test your new functionality:

Create a new object instance for Vera.

```
vera = Athlete('Vera Vi')
```

```
vera.add_time('1:31')
```

← Add a single timing value.

```
print(vera.top3())
```

← This will display a list with only one value in it: 1:31.

```
vera.add_times(['2:22', '1-21', '2:22'])
```

← Add three more timing values.

```
print(vera.top3())
```

← The top 3 timing scores are now: 1:21, 1:31 and 2:22.



Amend your code with the updated version of your `Athlete` class *before* proceeding with this Test Drive.



Test Drive

After running your existing program, try out your test code in the IDLE shell to confirm that everything is working as expected.

```

Python Shell
>>> ===== RESTART =====
>>>
James Lee's fastest times are: ['2.01', '2.16', '2.22']
Julie Jones's fastest times are: ['2.11', '2.23', '2.59']
Mikey McManus's fastest times are: ['2.22', '2.31', '2.38']
Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']
>>>
>>> vera = Athlete('Vera Vi')
>>> vera.add_time('1.31')
>>> print(vera.top3())
['1.31']
>>>
>>> vera.add_times(['2.22', '1-21', '2:22'])
>>> print(vera.top3())
['1.21', '1.31', '2.22']
>>>

```

← As expected.

← Create a new athlete.

← Add one timing value.

← Display the top three times (there's only one, so that's all you see).

← Add three more timing values.

← Display the top three times (which, now, makes a little more sense).

Ln: 179 Col: 4

Great: it worked.

You've **packaged** your code with your data and created a custom class from which you can create objects that share behaviors. And when extra functionality is required, **add more methods** to implement the required functionality.

By **encapsulating** your athlete code and data within a custom class, you've created a much more **maintainable** piece of software. You will thank yourself for doing this when, in six months, you need to amend your code.

Well done. This is really coming along!



Emmm...maybe I'm missing something, but isn't your `Athlete` class wasteful? I mean, you've extended it with functionality that's **already** in lists, which feels a little like reinventing the wheel to me...

Yes, your `Athlete` class is much like a list.

Your `Athlete` class does indeed behave like a **list** *most of the time*, and you've added methods to expose some list functionality to the users of your class. But it's true: *you are* reinventing the wheel here. Your `add_time()` method is a thin wrapper around the list `append()` method and your `add_times()` method is list's `extend()` method in disguise.

In fact, your `Athlete` class only differs from Python's list due to the inclusion of the `name` and `dob` object attributes.



Wouldn't it be dreamy if there were a way to extend a built-in class with custom attributes? But I know it's just a fantasy...

Inherit from Python's built-in list

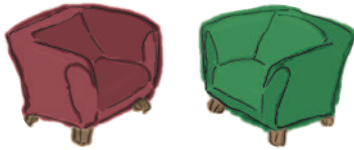
Python's **class** lets you create a custom class *from scratch*, just like you did with your `Athlete` class. However, **class** also lets you create a class by *inheriting* from any other existing class, including Python's built-in data structure classes that provide you with `list`, `set`, and `dict`. Such classes are referred to as *subclasses*.

What's really nice is that when you inherit from an existing class (such as `list`), you are given all of the existing functionality for free.

As your existing class is really nothing more than a **list** with added attributes, perhaps a better design is to kill off your `Athlete` class and replace it with a class that inherits from the built-in `list` class? It's certainly worth considering, isn't it?



Fireside Chats



Tonight's talk: **Inheritance, a.k.a. He looks just like his father.**

Custom Class:

Programmers like me because they get to control *everything* in their code...and you know programmers: *they love to code*.

Design! Phooey! Real programmers eat, sleep, dream, snore, and exhale code. All that design talk is for people who *can't* code!

No, no, no: you're not listening. It's all done with control. When you build everything from the ground up, you're in control, as *it's all your code*.

Of course, especially when there are custom requirements to be taken into consideration. In that case, a brand-spanking new custom class is the only way to go.

Yeah, right...it's a win-win for you, not me.

I guess so, although I'm still a fan of custom code...

Inherited Class:

Yes, they do. But sometimes writing everything from scratch is not the best design decision.

Is it really? So, you're saying it's much better to do everything from scratch and repeat the work of others, because your way is the best way. Are you serious?!?

And you're happy to reinvent the wheel, even though someone else solved that problem eons ago?

Not if you can extend someone else's class to handle your custom requirements. That way, you get the best of both worlds: inherited functionality (so you're not reinventing the wheel) together with the custom bits. It's a win-win situation.

But it's not about us: it's to do with making the life of the programmer easier, even the ones that live to code, right?



An IDLE Session

Let's see what's involved in inheriting from Python's built-in `list` class. Working in IDLE's shell, start by creating a custom list derived from the built-in `list` class that also has an attribute called `name`:

```
>>> class NamedList(list):
    def __init__(self, a_name):
        list.__init__([])
        self.name = a_name
```

Provide the name of the class that this new class derives from.

Initialize the derived from class, and then assign the argument to the attribute.

With your `NamedList` class defined, use it to create an object instance, check the object's type (using the `type()` BIF), and see what it provides (using the `dir()` BIF):

```
>>> johnny = NamedList("John Paul Jones")
>>> type(johnny)
<class '__main__.NamedList'>
>>> dir(johnny)
['_add_', '_class_', '_contains_', '_delattr_', '_delitem_', '_dict_', '_doc_',
'_eq_', '_format_', '_ge_', '_getattr_', '_getitem_', '_gt_', '_hash_',
'_iadd_', '_imul_', '_init_', '_iter_', '_le_', '_len_', '_lt_', '_module_',
'_mul_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_', '_reversed_',
'_rmul_', '_setattr_', '_setitem_', '_sizeof_', '_str_', '_subclasshook_',
'_weakref_', 'append', 'count', 'extend', 'index', 'insert', 'name', 'pop', 'remove',
'reverse', 'sort']
```

Create a new "NamedList" object instance.

Yes, "johnny" is a "NamedList".

"johnny" can do everything a list can, as well as store data in the "name" attribute.

Use some of the functionality supplied by the `list` class to add to the data stored in `johnny`:

```
>>> johnny.append("Bass Player")
>>> johnny.extend(['Composer', 'Arranger', 'Musician'])
>>> johnny
['Bass Player', 'Composer', 'Arranger', 'Musician']
>>> johnny.name
'John Paul Jones'
```

Add data to the "NamedList" using the methods provided by the list built in.

Access the list data, as well as the attribute data.

Because `johnny` is a **list**, it's quite OK to do list-type things to it:

```
>>> for attr in johnny:
    print(johnny.name + " is a " + attr + ".")
```

"johnny" is like any other list, so feel free to use it wherever you'd use a list.

```
John Paul Jones is a Bass Player.
John Paul Jones is a Composer.
John Paul Jones is a Arranger.
John Paul Jones is a Musician.
```

Confirmation: John's a busy boy. ☺

```
class Athlete:
    def __init__(self, a_name, a_dob=None, a_times=[]):
        self.name = a_name
        self.dob = a_dob
        self.times = a_times

    def top3(self):
        return(sorted(set([sanitize(t) for t in self.times]))[0:3])

    def add_time(self, time_value):
        self.times.append(time_value)

    def add_times(self, list_of_times):
        self.times.extend(list_of_times)
```

[illegible]

.....



Exercise Solution

Here is the code for the now defunct `Athlete` class. In the space provided below, you were to rewrite this class to inherit from the built-in `list` class. You were to call your new class `AthleteList`, as well as provide a few lines of code to exercise your new class:

```
class Athlete:
    def __init__(self, a_name, a_dob=None, a_times=[]):
        self.name = a_name
        self.dob = a_dob
        self.times = a_times

    def top3(self):
        return(sorted(set([sanitize(t) for t in self.times]))[0:3])

    def add_time(self, time_value):
        self.times.append(time_value)

    def add_times(self, list_of_times):
        self.times.extend(list_of_times)
```

The class name has changed.

Inherit from the built-in `list` class.

```
class AthleteList(list):
    def __init__(self, a_name, a_dob=None, a_times=[]):
```

Nothing new here... this code is very similar to the "NamedList" init code.

```
list.__init__([])
self.name = a_name
self.dob = a_dob
self.extend(a_times)
```

```
def top3(self):
    return(sorted(set([sanitize(t) for t in self]))[0:3])
```

Use the new class's name.

```
vera = AthleteList('Vera Vi')
```

```
vera.append('1.31')
print(vera.top3())
vera.extend(['2.22', '1-21', '2:22'])
print(vera.top3())
```

Now that you're inheriting from the built-in `list`, you can use its methods to get your work on.

This code does a good job of exercising your new class.

These methods aren't needed anymore.

The data itself is the timing data, so the "times" attribute is gone.



In your code, replace your `Athlete` class code with your new `AthleteList` class code, and don't forget to change `get_coach_data()` to return an `AthleteList` object instance as opposed to an `Athlete` object instance.

there are no Dumb Questions

Q: Sorry...but not three minutes ago you were telling me not to expose the inner workings of my class to its users, because that was fundamentally a bad idea. Now you're doing the exact opposite! What gives?

A: Well spotted. In this particular case, it's OK to expose the fact that the class is built on top of `list`. This is due to the fact that the class is deliberately called `AthleteList` to distinguish it from the more generic `Athlete` class. When programmers see the word "list" in a class name, they are likely to expect the class to work like a `list` and then some. This is the case with `AthleteList`.

Q: And I can inherit from any of the built-in types?

A: Yes.

Q: What about inheriting from more than one class...does Python support multiple inheritance?

A: Yes, but it's kind of scary. Refer to a good Python reference text for all the gory details.

Q: Can I inherit from my own custom classes?

A: Of course, that's the whole idea. You create a generic class that can then be "subclassed" to provide more specific, targeted functionality.

Q: Can I put my class in a module file?

A: Yes, that's a really good idea, because it lets you share your class with many of your own programs and with other programmers. For instance, if you save your `AthleteList` class to a file called `athletelist.py`, you can import it into your code using this line of code:

```
from atletelist import AthleteList
```

then use the class as if it was defined in your current program. And, of course, if you create a really useful class, pop it into its own module and upload it to PyPI for the whole world to share.



Test Drive

One last run of your program should confirm that it's working to specification now. Give it a go in IDLE to confirm.

```
coach6.py - /Users/barryp/HeadFirstPython/chapter6/coach6.py

def sanitize(time_string):
    if '-' in time_string:
        splitter = '-'
    elif ':' in time_string:
        splitter = ':'
    else:
        return(time_string)
    (mins, secs) = time_string.split(splitter)
    return(mins + '.' + secs)

class AthleteList(list):
    def __init__(self, a_name, a_dob=None, a_times=[]):
        list.__init__([])
        self.name = a_name
        self.dob = a_dob
        self.extend(a_times)

    def top3(self):
        return(sorted(set([sanitize(t) for t in self]))[0:3])

def get_coach_data(filename):
    try:
        with open(filename) as f:
            data = f.readline()
            templ = data.strip().split(',')
            return(AthleteList(templ.pop(0), templ.pop(0), templ))
    except IOError as ioerr:
        print('File error: ' + str(ioerr))
        return(None)

james = get_coach_data('james2.txt')
julie = get_coach_data('julie2.txt')
mikey = get_coach_data('mikey2.txt')
sarah = get_coach_data('sarah2.txt')

print(james.name + "'s fastest times are: " + str(james.top3()))
print(julie.name + "'s fastest times are: " + str(julie.top3()))
print(mikey.name + "'s fastest times are: " + str(mikey.top3()))
print(sarah.name + "'s fastest times are: " + str(sarah.top3()))
```

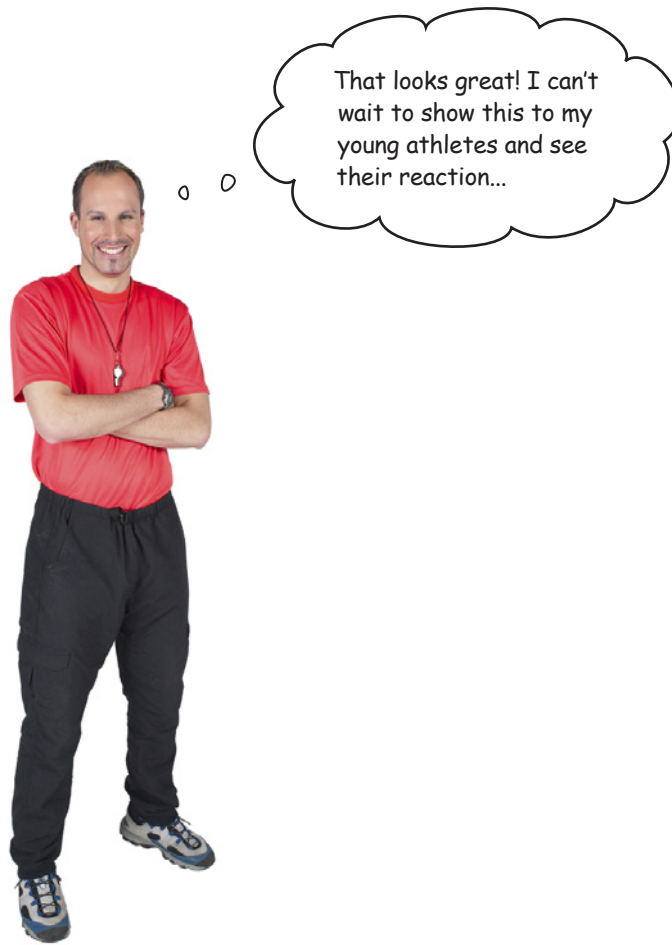
Your entire
program now
produces the
output the
coach wants.

```
Python Shell

>>> ===== RESTART =====
>>>
James Lee's fastest times are: ['2.01', '2.16', '2.22']
Julie Jones's fastest times are: ['2.11', '2.23', '2.59']
Mikey McManus's fastest times are: ['2.22', '2.31', '2.38']
Sarah Sweeney's fastest times are: ['2.18', '2.21', '2.22']
>>> |
```

Ln: 306 Col: 4

Coach Kelly is impressed



By basing your class on built-in functionality, you've leveraged the power of Python's data structures while providing the custom solution your application needs.

You've **engineered** a much more *maintainable solution* to Coach Kelly's data processing needs.

Good job!



Your Python Toolbox

You've got Chapter 6 under your belt and you've added some key Python techniques to your toolbox.

Python Lingo

- "Dictionary" – a built-in data structure that allows you to associate data values with keys.
- "Key" – the look-up part of the dictionary.
- "Value" – the data part of the dictionary (which can be any value, including another data structure).

More Python Lingo

- "self" – a method argument that always refers to the current object instance.



BULLET POINTS

- Create an empty **dictionary** using the `dict()` factory function or using `{}`.
- To access the value associated with the key `Name` in a dictionary called `person`, use the familiar square bracket notation: `person['Name']`.
- Like **list** and **set**, a Python's dictionary dynamically grows as new data is added to the data structure.
- Populate a dictionary as you go:

```
new_d = {} or new_d = dict()
and then
d['Name'] = 'Eric Idle'
or do the same thing all in the one go:
new_d = {'Name': 'Eric Idle'}
```
- The `class` keyword lets you define a class.
- Class **methods** (your code) are defined in much the same way as functions, that is, with the `def` keyword.
- Class **attributes** (your data) are just like variables that exist within object instances.
- The `__init__()` method can be defined within a class to initialize object instances.
- Every method defined in a class must provide `self` as its first argument.
- Every attribute in a class must be prefixed with `self.` in order to associate it data with its instance.
- Classes can be built from scratch or can **inherit** from Python's built-in classes or from other custom classes.
- Classes can be put into a Python module and **uploaded** to PyPI.